

陕西科技大学 试题纸 A

课程 算法分析与设计 学期

班级 学号 姓名

题号		一	三	四	五	六	七	八	九	十	总分
得分											
阅卷人											

一、单项选择题（每小题 2 分，共 20 分）

1. 算法分析的两个主要方面是 (A)
A. 空间复杂度和时间复杂度 B. 正确性和简单性
C. 可读性和文档性 D. 数据复杂度和程序复杂度
2. 计算机算法指的是 (C)
A. 计算方法 B. 排序方法
C. 解决问题的方法和过程 D. 调度方法
3. 多阶段决策问题就是要在可以选择的那些策略中间选取一个 (A) 策略使在预定的标准下达到最好的效果。
A. 最优 B. 最差 C. 平衡 D. 任意
4. 根据排序元素所在位置的不同，排序分 (D) ~~A~~
A. 内排序和外排序 B. 首排序和尾排序
C. 顺序排序和逆序排序 D. 堆排序和栈排序
5. 直接或间接的调用自身的算法称为 (B)
A. 贪心算法 B. 递归算法
C. 迭代算法 D. 动态规划算法
6. 与分治法不同的是，适合于用动态规划求解的问题 (A)
A. 经分解得到子问题往往不是互相独立的
B. 经分解得到子问题往往是互相独立的
C. 经分解得到子问题往往是互相交叉的



- D. 经分解得到了问题往往是任意的
7. 二分搜索算法的基本思想是将 n 个元素分成个数大致相同的两半, 取 $a[n/2]$ 与 x 进行比较: 如果 (A) 则只要在数组 a 的左半部继续搜索 x 。
- A. $x < a[n/2]$ B. $x > a[n/2]$ C. $x \leq a[n/2]$ D. $x \geq a[n/2]$
8. 活动安排问题就是在所给的活动集合中, 选出 (C) 的相容活子集。
- A. 最小 B. 任意 C. 最大 D. 一个
9. 在对问题的解空间树进行搜索的方法中一个活结点最多有一次机会成为活结点的是 (B)
- A. 回溯法 B. 分支限界法 C. 回溯法和分支限界法 D. 回溯法求解子集树问题
10. 适用动态规划的问题必须满足 (D)
- A. 最优化原理 B. 无前效性 C. 最优化原理和后效性 D. 最优化原理和无后效性

二、简答题（每小题 10 分，共 40 分）

- 简述分治法的基本思想。
- 动态规划的基本思想。
- 阐述分支限界法的搜索策略。
- 简述蒙特卡罗算法、拉斯维加斯算法、舍伍德算法的设计思想和特点。

三、算法填空题（每小题 10 分，共 10 分）

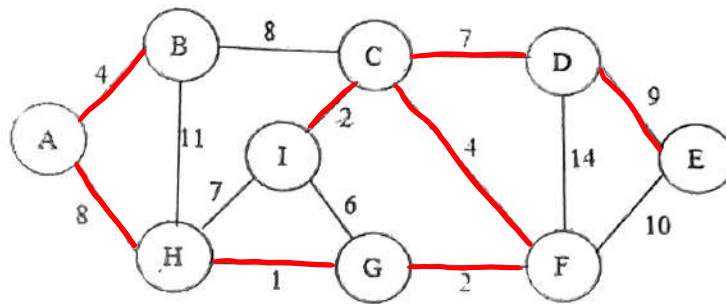
- 排序算法分为初等排序和高等排序, 列举了三种排序算法, 请填写不同测试数据集下, 表现最好的算法, 并说明原因。

测试数据集	插入排序	归并排序	基排序
字符串 10k 乱序	✓		
整型 1M 逆序		✓	
字符串 10M 乱序			✓

四、算法分析与设计题（每小题 10 分，共 20 分）

- 对下图所示的连通网络 G , 用克鲁斯卡尔 (Kruskal) 算法求 G 的最小生成树 T , 请写出在算法执行过程中, 依次加入树 T 的边集 TE 中的边, 代码如下所示, 分析算法思想及时间复杂度。





带权连通图

```
package suanfa;
import java.util.Arrays;
public class Main {
    static class Graph {
        Edge[] edges;
        int[][] arr;
    }
    static class Edge implements Comparable<Edge> {
        int begin;
        int end;
        int weight;
        @Override
        public int compareTo(Edge o) {
            return this.weight - o.weight;
        }
    }

    public static void kruskal(Graph graph) {
        // 从小到大按权值排好序的 edges
        Edge[] edges = graph.edges;
        int[][] arr = graph.arr;
        int[] parent = new int[7];
        // 顶点的编号为0-6
        for (int i = 0; i < 7; i++) {
            parent[i] = 0;
        }
        for (int i = 0; i < edges.length; i++) {
            Edge edge = edges[i];
            int rootOfBegin = findParentRoot(edge.begin, parent);
            int rootOfEnd = findParentRoot(edge.end, parent);
            if (rootOfBegin != rootOfEnd) {
                System.out.println(String.format("(%d,%d)->%d",
                    rootOfBegin, rootOfEnd, edge.weight));
                parent[rootOfBegin] = rootOfEnd;
            }
        }
    }

    private static int findParentRoot(int target, int[] parent) {
        while (parent[target] > 0) {
            target = parent[target];
        }
        return target;
    }

    public static void main(String[] args) {
        // 初始化
        Graph graph = new Graph();
        int[][] arr = new int[7][7];
        for (int i = 0; i < 7; i++) {
            for (int j = 0; j < 7; j++) {
                if (i == j)
                    arr[i][j] = 0;
                else {
                    arr[j][i] = Integer.MAX_VALUE;
                }
            }
        }
    }
}
```



```

    }
    }
    arr[0][1] = 7;
    arr[0][3] = 5;
    arr[1][2] = 8;
    arr[1][3] = 9;
    arr[1][4] = 7;
    arr[2][4] = 5;
    arr[3][4] = 15;
    arr[3][5] = 6;
    arr[4][5] = 8;
    arr[4][6] = 9;
    arr[5][6] = 11;
    for (int i = 0; i < 7; i++) {
        for (int j = i; j < 7; j++) {
            arr[j][i] = arr[i][j];
        }
    }
    graph.arr = arr;
    int k = 0;
    Edge[] edges = new Edge[11];
    for (int i = 0; i < edges.length; i++) {
        Edge edge = new Edge();
        edges[i] = edge;
    }
    for (int i = 0; i < 6; i++) {
        for (int j = i + 1; j < 7; j++) {
            if (arr[i][j] < Integer.MAX_VALUE) {
                edges[k].begin = i; // 编号较小的结点为首
                edges[k].end = j; // 编号较大的结点为尾
                edges[k].weight = arr[i][j];
                k++;
            }
        }
    }
    graph.edges = edges;
    Arrays.sort(edges);
    // 初始化结束
    kruskal(graph);
}
}

```

2. 对下列各组函数 $f(n)$ 和 $g(n)$, 确定 $f(n) = O(g(n))$ 或 $f(n) = \Omega(g(n))$ 或 $f(n) = \theta(g(n))$, 并简要说明理由。

- | | |
|-----------------------|-----------------|
| (1) $f(n)=2^n$; | $g(n)=n!$ |
| (2) $f(n)=\sqrt{n}$; | $g(n)=\log n^2$ |
| (3) $f(n)=100$; | $g(n)=\log 100$ |
| (4) $f(n)=n^3$; | $g(n)=3^n$ |
| (5) $f(n)=3^n$; | $g(n)=2^n$ |



陕西科技大学 试题纸 A 答案和评分标准

课程 算法分析与设计

学期

一、单项选择题（每小题 2 分，共 20 分）

答案：

1. A 2. C 3. A 4. A 5. B 6. A 7. A 8. C 9. B 10. D

二、简答题（每小题 10 分，共 40 分）

1. 简述分治法的基本思想。

答：将一个规模为 n 的问题分解为 k 个规模较小的子问题，这些子问题互相独立且与原问题相同。递归地解这些子问题，然后将各个子问题的解合并得到原问题的解。

2. 动态规划的基本思想

动态规划的实质是分治思想和解决冗余，因此，动态规划是一种将问题实例分解为更小的、相似的子问题，并存储子问题的解而避免计算重复的子问题，以解决最优化问题的算法策略。

由此可知，动态规划法与分治法和贪心法类似，它们都是将问题实例归纳为更小的、相似的子问题，并通过求解子问题产生一个全局最优解。

贪心法的当前选择可能要依赖已经作出的所有选择，但不依赖于有待于做出的选择和子问题。因此贪心法自顶向下，一步一步地作出贪心选择：

而分治法中的各个子问题是独立的（即不包含公共的子问题），因此一旦递归地求出各子问题的解后，便可自下而上地将子问题的解合并成问题的解。

不足之处：如果当前选择可能要依赖子问题的解时，则难以通过局部的贪心策略达到全局最优解；如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题。

解决上述问题的办法是利用动态规划。该方法主要应用于最优化问题，这类问题会有多种可能的解，每个解都有一个值，而动态规划找出其中最优（最大或最小）值的解。若存在若干个取最优值的解的话，它只取其中的一个。在求解过程中，该方法也是通过求解局部子问题的解达到全局最优解，但与分治法和贪心法不同的是，动态规划允许这些子问题不独立，（亦即各子问题可包含公共的子问题）也允许其



通过自身子问题的解作出选择，该方法对每一个子问题只解一次，并将结果保存起来，避免每次碰到时都要重复计算。

因此，动态规划法所针对的问题有一个显著的特征，即它所对应的子问题树中的子问题呈现大量的重复。动态规划法的关键就在于，对于重复出现的子问题，只在第一次遇到时加以求解，并把答案保存起来，让以后再遇到时直接引用，不必重新求解。

3. 阐述分支限界法的搜索策略。

在扩展结点处，先生成其所有的儿子结点（分支），然后再从当前的活结点表中选择下一个扩展结点。为了有效地选择下一扩展结点，加速搜索的进程，在每一个活结点处，计算一个函数值（限界），并根据函数值，从当前活结点表中选择一个最有利的结点作为扩展结点，使搜索朝着解空间上有最优解的分支推进，以便尽快地找出一个最优解。

4. 简述蒙特卡罗算法、拉斯维加斯算法、舍伍德算法的设计思想和特点。

蒙特卡罗 (Monte Carlo) 算法在一般情况下可以保证对问题的所有实例都以高概率给出正确解，但是通常无法判定一个具体解是否正确。设 p 是一个实数，且 $1/2 < p < 1$ 。如果一个蒙特卡罗算法对于问题的任一实例得到正确解的概率不小于 p ，则称该蒙特卡罗算法是 p 正确的，且称 $p-1/2$ 是该算法的优势。如果对于同一实例，蒙特卡罗算法不会给出 2 个不同的正确解答，则称该蒙特卡罗算法是一致的。

拉斯维加斯 (Las Vegas) 算法的一个显著特征是它所作的随机性决策有可能导致算法找不到所需的解。

```
void obstinate(Object x, Object y)
{
    // 反复调用拉斯维加斯算法 LV(x, y)，直到找到问题的一个解 y
    bool success= false;
    while (!success) success=lv(x, y);
}
```

设 $p(x)$ 是对输入 x 调用拉斯维加斯算法获得问题的一个解的概率。一个正确的拉斯维加斯算法应该对所有输入 x 均有 $p(x) > 0$ 。

舍伍德算法设 A 是一个确定性算法，当它的输入实例为 x 时所需的计算时间记为 $t_A(x)$ 。设 X_n 是算法 A 的输入规模为 n 的实例的全体，则当问题的输入规模为 n 时，算



$$t_A(n) = \sum_{x \in X_n} t_A(x) / |X_n|$$

法 A 所需的平均时间为

这显然不能排除存在 $x \in X_n$ 使得

的可能性。希望获得一个概率算法 B，使得对问题的输入规模为 n 的每一个实例均有

$$t_B(x) = t_A(n) + s(n)$$

这就是舍伍德算法设计的基本思想。当 $s(n)^t$ 与 $t_A(n)$ 相比可忽略时，舍伍德算法可获得良好的平均性能。

二、填空题（每小题 10 分，共 10 分）

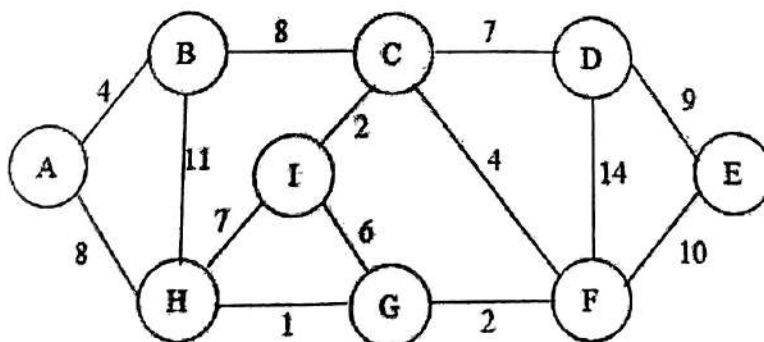
1. 排序算法分为初等排序和高等排序，列举了一种排序算法，请填写不同测试数据集下，表现最好的算法，并说明原因。

测试数据集	插入排序	归并排序	基排序
字符串 10k 乱序	最快		
整型 1M 逆序		最快	
字符串 10M 乱序			最快

答：插入排序输入初等排序，在小数据中表现好；归并排序在较大数据下的最差情况最稳定；基排序对字符串排序的平均效率高于归并排序。

四、算法分析题（每小题 15 分，共 30 分）

1. 对下图所示的连通网络 G，用克鲁斯卡尔(Kruskal)算法求 G 的最小生成树 T，请写出在算法执行过程中，依次加入树 T 的边集 TE 中的边，代码如下所示，分析算法思想及时间复杂度。



带权连通图

```
package suanfa;
import java.util.Arrays;
public class Main {
    static class Graph {
        Edge[] edges;
```




```

        int[][] arr;
    }
    static class Edge implements Comparable<Edge> {
        int begin;
        int end;
        int weight;
        @Override
        public int compareTo(Edge o) {
            return this.weight - o.weight;
        }
    }

    public static void kruskal(Graph graph) {
        // 从小到大按权值排好序的 edges
        Edge[] edges = graph.edges;
        int[][] arr = graph.arr;
        int[] parent = new int[7];
        // 顶点的编号为 0-6
        for (int i = 0; i < 7; i++) {
            parent[i] = 0;
        }
        for (int i = 0; i < edges.length; i++) {
            Edge edge = edges[i];
            int rootOfBegin = findParentRoot(edge.begin, parent);
            int rootOfEnd = findParentRoot(edge.end, parent);
            if (rootOfBegin != rootOfEnd) {
                System.out.println(String.format("(%d,%d)->%d",
                    rootOfBegin, rootOfEnd, edge.weight));
                parent[rootOfBegin] = rootOfEnd;
            }
        }
    }

    private static int findParentRoot(int target, int[] parent) {
        while (parent[target] > 0) {
            target = parent[target];
        }
        return target;
    }

    public static void main(String[] args) {
        // 初始化
        Graph graph = new Graph();
        int[][] arr = new int[7][7];
        for (int i = 0; i < 7; i++) {
            for (int j = 0; j < 7; j++) {
                if (i == j)
                    arr[i][j] = 0;
                else {
                    arr[j][i] = Integer.MAX_VALUE;
                }
            }
        }
        arr[0][1] = 7;
        arr[0][3] = 5;
        arr[1][2] = 8;
        arr[1][3] = 9;
        arr[1][4] = 7;
        arr[2][4] = 5;
        arr[3][4] = 15;
        arr[3][5] = 6;
        arr[4][5] = 8;
        arr[4][6] = 9;
        arr[5][6] = 11;
        for (int i = 0; i < 7; i++) {
            for (int j = i; j < 7; j++) {
                arr[j][i] = arr[i][j];
            }
        }
        graph.arr = arr;
    }
}

```




```

int k = 0;
Edge[] edges = new Edge[11];
for(int i=0; i<edges.length; i++){
    Edge edge = new Edge();
    edges[i] = edge;
}
for(int i = 0; i < 6; i++){
    for(int j = i + 1; j < 7; j++){
        if(arr[i][j] < Integer.MAX_VALUE){
            edges[k].begin = i; // 编号较小的结点为首
            edges[k].end = j; // 编号较大的结点为尾
            edges[k].weight = arr[i][j];
            k++;
        }
    }
}
graph.edges = edges;
Arrays.sort(edges);
// 初始化结束
kruskal(graph);
}
}

```

答:

贪心策略是每次都在连接两个不同连通分量的边中选权值最小的边。

基本思想: 首先将图中所有顶点都放到生成树中, 然后每次都在连接两个不同连通分量的边中选权值最小的边, 将其放入生成树中, 直到生成树中有 $n-1$ 条边。

时间复杂度为: $O(n^2 \lg n)$

2. 对下列各组函数 $f(n)$ 和 $g(n)$, 确定 $f(n) = O(g(n))$ 或 $f(n) = \Omega(g(n))$ 或 $f(n) = \theta(g(n))$, 并简要说明理由。

- | | |
|-----------------------|-----------------|
| (1) $f(n)=2^n$; | $g(n)=n!$ |
| (2) $f(n)=\sqrt{n}$; | $g(n)=\log n^2$ |
| (3) $f(n)=100$; | $g(n)=\log 100$ |
| (4) $f(n)=n^3$; | $g(n)=3^n$ |
| (5) $f(n)=3^n$; | $g(n)=2^n$ |

答:

- | | |
|---------------------------|---------------------------|
| (1) $f(n) = O(g(n))$ | 因为 $g(n)$ 的阶比 $f(n)$ 的阶高。 |
| (2) $f(n) = \Omega(g(n))$ | 因为 $g(n)$ 的阶比 $f(n)$ 的阶低。 |
| (3) $f(n) = \theta(g(n))$ | 因为 $g(n)$ 与 $f(n)$ 同阶。 |
| (4) $f(n) = O(g(n))$ | 因为 $g(n)$ 的阶比 $f(n)$ 的阶高。 |
| (5) $f(n) = \Omega(g(n))$ | 因为 $g(n)$ 的阶比 $f(n)$ 的阶低。 |

